CODE SPORT

In this month's column, we shift our focus to mobile application development and discuss the issue of energy consumption of mobile apps. We also briefly cover how a layman can quickly learn to develop Android apps.

For the past few months, we have been discussing information retrieval and natural language processing as well as the algorithms associated with them. In this month's column, we take a break from our discussion of NLP and, instead, focus on mobile application development. We discuss some of the recent research work on the energy consumed by mobile apps. Then we move on to mobile app development, which has been growing at a furious pace over the past few years and has attracted many non-software developers to its fold, including school students. Learning to develop mobile apps using open source software is quite easy. In this month's column, we have one of our student readers, Ankith Subramanya, who is a Class XI student from The International School, Bengaluru, share his experience of learning to develop an Android app.

Energy consumption in mobile apps

Smartphones run complete operating systems tailored for the mobile device, and support a wide variety of additional hardware devices such as cameras, GPS, a light sensor, accelerometer, gyroscope, compass, etc. Along with such a rich plethora of 'exotic' hardware, smartphones also need to be extremely lightweight and portable. This requires that smartphone batteries must be small and lightweight. This, in turn, implies that energy consumption in smartphones needs to be as low as possible to conserve battery life. Hence, energy consumption by mobile apps needs to be tracked and managed to extend the battery life of smartphones.

Smartphones have become open platforms for hosting a variety of applications. Given the rapid pace of mobile app development, more and more apps are being developed, downloaded and installed on to the device, without their performance, security and energy characteristics being validated and documented. This can lead to poorly written apps causing an abnormal drain on the device's battery.

Given a smartphone with a fully charged battery, the user expects that the device should be usable for a certain number of hours without requiring a battery recharge. The user is aware that prolonged use of certain hardware such as the camera, GPS, etc, can shorten the period before the next recharge. However, in the course of using normal apps, such 'Abnormal Battery Drain' (ABD) issues are not expected to occur. When an ABD issue occurs, it is difficult for normal users to figure out what it is caused by. Some of the potential causes could be configuration changes; a hardware or software malfunction which keeps a power-hungry hardware component such as the camera always on, even when not in use; an app upgrade, etc. Hence, intelligent tools are needed to diagnose ABD issues. One such tool is a research project called eDoctor, which is described in the following research paper *http://cseweb.ucsd.edu/~voelker/pubs/edoctor-nsdi13.pdf*. eDoctor records various events such as configuration changes, app installations and app upgrades; and it monitors app phases. Using this information, it tries to pinpoint what is responsible for the ABD.

Given the importance of conserving battery life, smartphone OSs are designed to turn off hardware components the moment they detect that they are not in use. Smartphones employ aggressive CPU/screen sleeping policies. Such measures can result in issues for apps which are in the middle of communicating with an external server and are waiting for data to be returned from the server. In such cases, the device should not be put to sleep. Hence, smartphone OSs export explicit 'wakelock' APIs, which can be used